BITS Pilani

Deep Neural Network
Module # 8: Attention Mechanism

Seetha Parameswaran
WILPD - CSIS

innovate achieve lead

The instructor is gratefully acknowledging
the authors who made their course
materials freely available online.
This deck is prepared by Seetha Parameswaran.

Work Integrated Learning Programmes

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Table of Contents

1. Encoder-Decoder Foundation
2. Introduction to Attention Mechanisms
3. Query-Key-Value Framework
4. Attention Scoring Functions
5. Bahdanau Attention for Seq2Seq
6. Types of Attention
7. Self-Attention & Positional Encoding
8. Summary & Key Takeaways

Work Integrated Learning Programmes

WHY STUDY ENCODER-DECODER?

Many NLP tasks require variable-length sequence-to-sequence mapping:

- Sentiment Analysis:
  - Input: "The phone battery lasts only 2 hours. Very disappointed."
  - Output: "Negative"
- Question Answering:
  - Input: Context + "What is the capital of India?"
  - Output: "New Delhi"
- Text Summarization:
  - Input: 500-word article
  - Output: 50-word summary

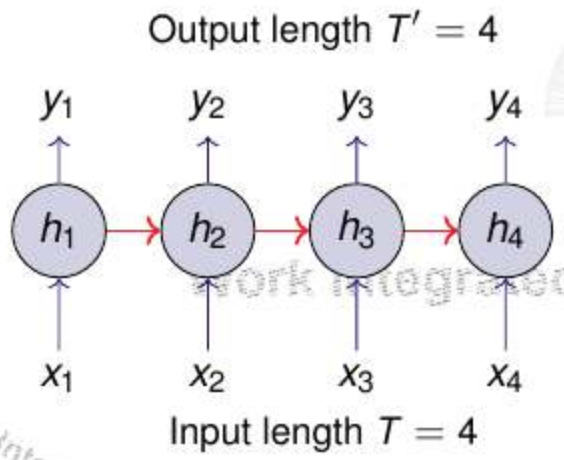
Key observation: Input length $T \neq$ Output length T' in general.

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

THE CHALLENGE: STANDARD RNN LIMITATIONS

Example: Question Answering fails with standard RNN

Standard RNN: Produces one output per input step (requires $T = T'$)



- Input: "India gained independence from Britain. What year was independence?" (10 words)
- Desired Output: "1947" (1 word)
- Problem: Standard RNN produces 10 outputs, not 1!

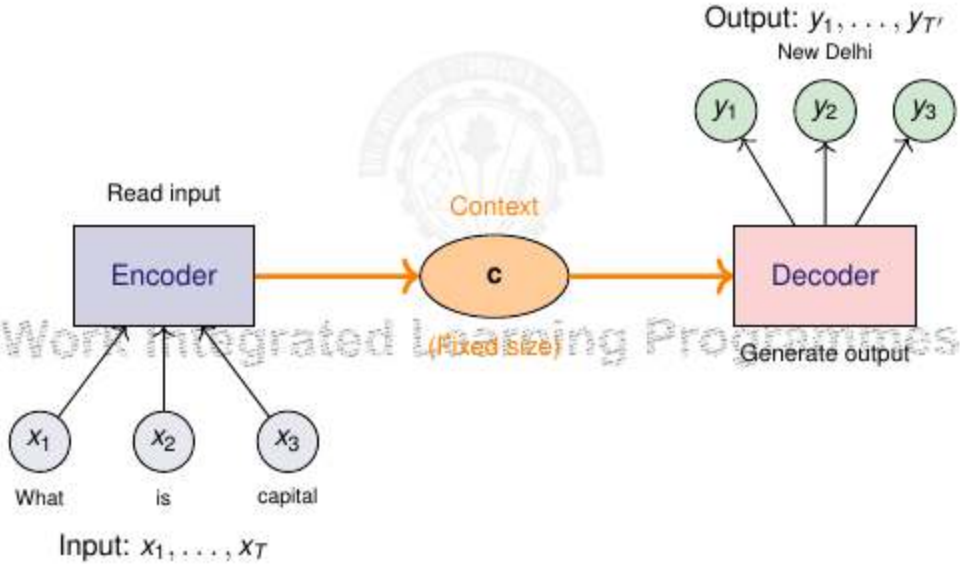
Solution needed: Separate **reading** phase from **generation** phase.

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Solution: Encoder-Decoder Architecture

Key Idea: Two-stage process

1. Encoder: Read and compress entire input into context vector **c**
2. Decoder: Generate output sequence from context, one token at a time



Flexibility: T (input length) can differ from T' (output length).

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

How Encoder-Decoder Works

- Phase 1: Encoding
  - Process input sequence with RNN:

$$h_t^{enc} = \phi(W_{hh}^{enc} h_{t-1}^{enc} + W_{hx}^{enc} x_t + b_h^{enc})$$

  - Extract context from final hidden state:

$$\mathbf{c} = h_T^{enc}$$

- Phase 2: Decoding
  - Initialize decoder with context:

$$h_0^{dec} = \mathbf{c}$$

  - Generate output sequence autoregressively:

$$h_t^{dec} = \phi(W_{hh}^{dec} h_{t-1}^{dec} + W_{hy}^{dec} y_{t-1} + b_h^{dec})$$

  - Output probability distribution:

$$P(y_t | y_{<t}, \mathbf{c}) = \text{softmax}(W_o h_t^{dec} + b_o)$$

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

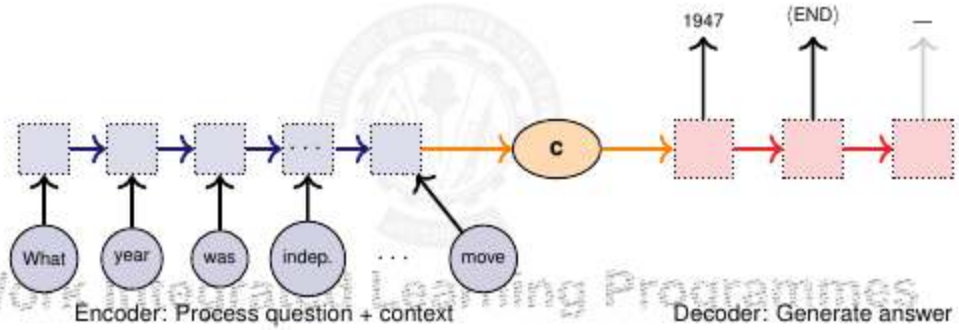
EXAMPLE: QUESTION ANSWERING WITH ENCODER-DECODER

- Task: Answer question from context

  Context: "India gained independence in 1947. Gandhi led the movement."

  Question: "What year was independence?"

  Answer: "1947"

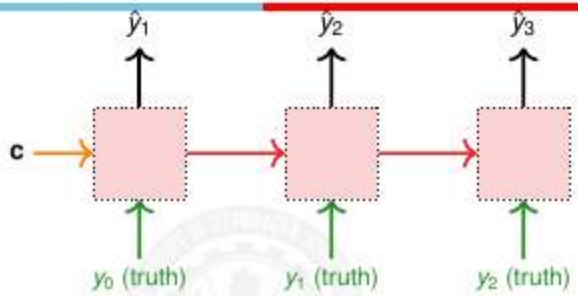


- Process:
  - Encoder reads entire input ($T \approx 15$ tokens)
  - Context **c** encodes: "answer is about year and independence"
  - Decoder generates: "1947" then ⟨END⟩ ($T' = 1$ meaningful token)

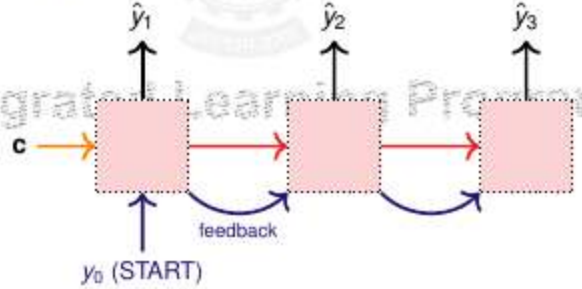
BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Training vs Inference

- Training (Teacher Forcing): Use ground truth as decoder input



- Inference (Free Running): Use model's own predictions

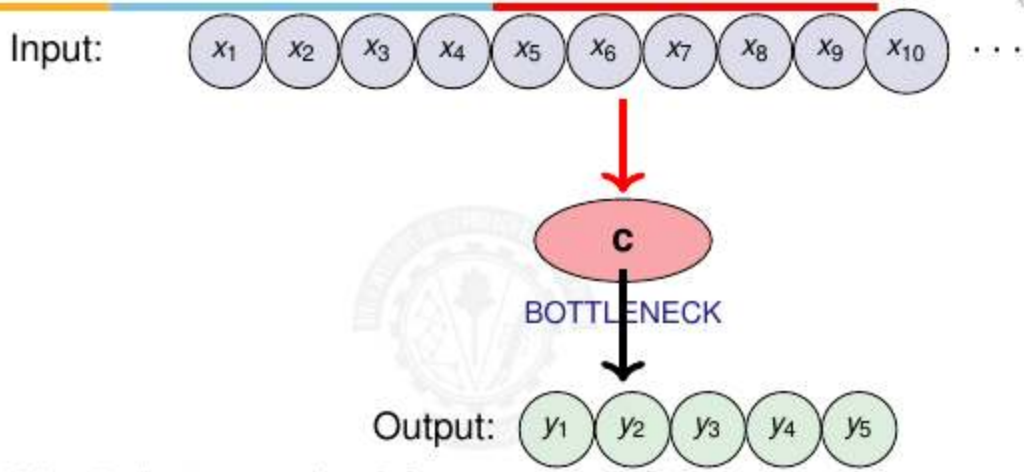


- Exposure Bias: Decoder trained on ground truth but tested on own predictions!

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

THE CRITICAL PROBLEM: CONTEXT BOTTLENECK

- All input information must pass through fixed-size vector **c**.



- Problems:
  1. Information loss: Long sequences → early tokens forgotten
  2. Fixed capacity: Vector size (e.g., 512-dim) limits information storage
  3. Uniform encoding: All input parts weighted equally, no selectivity
- Example: 50-word review compressed to 512 numbers, then generate sentiment!

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Bottleneck Impact: Concrete Example

- Task: Sentiment analysis of product review
- Input (30 words):

  "Ordered this laptop for work. Initial impressions were good. Fast delivery. However, after 2 weeks, battery drains in 2 hours. Screen flickers constantly. Very disappointed. Would not recommend."

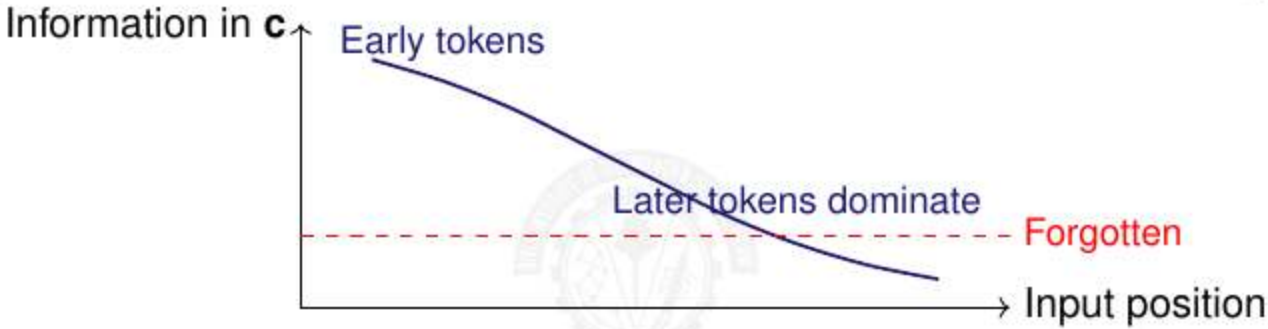
Early positive sentiment lost?
Key phrases "very disappointed" weakened?

- Issue: When generating "Negative", decoder needs to emphasize later parts, but **c** treats all parts equally!

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Why Fixed Context Fails for Long Sequences

Visualization of information loss:



Observations from research (Sutskever et al., 2014):

- Performance degrades significantly for sequences > 30 tokens
- First few words often lost by the time decoder generates output
- Model struggles with long-range dependencies

Question: How can we avoid this bottleneck?

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

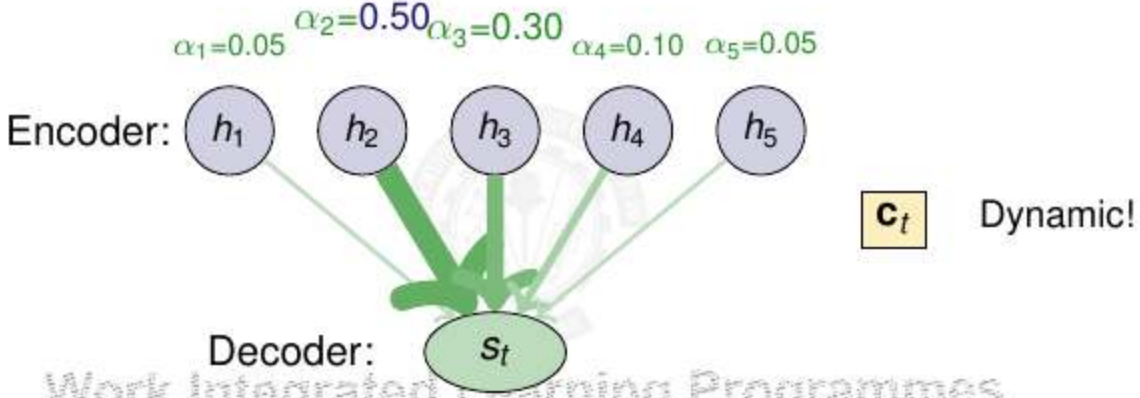
Table of Contents

1. Encoder-Decoder Foundation
2. Introduction to Attention Mechanisms
3. Query-Key-Value Framework
4. Attention Scoring Functions
5. Bahdanau Attention for Seq2Seq
6. Types of Attention
7. Self-Attention & Positional Encoding
8. Summary & Key Takeaways

Work Integrated Learning Programmes

THE SOLUTION: ATTENTION MECHANISMS

- Core Insight: Instead of single fixed context, let decoder selectively attend to relevant input parts at each step.



- Key Differences from Encoder-Decoder:
  - Before: Single context $\mathbf{c}$ for entire decoding
  - After: Different context $\mathbf{c}_t$ at each decoder step t
  - Weights α_i learned automatically from data
  - Focuses on relevant input positions dynamically

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

ATTENTION

Definition

Attention is a mechanism that computes a weighted sum of values based on the similarity between a query and keys.

- Components:
  - Query ($\mathbf{q}$): What we're looking for (decoder state)
  - Keys ($\{\mathbf{k}_i\}$): What's available to match (encoder states)
  - Values ($\{\mathbf{v}_i\}$): Actual content to retrieve (encoder states)
- Mathematical Form:

$$\text{Attention}(\mathbf{q}, \{\mathbf{k}_i, \mathbf{v}_i\}_{i=1}^{n}) = \sum_{i=1}^{n} \alpha_i \mathbf{v}_i$$

$$\text{Attention weights} \quad \alpha_i = \frac{\exp(\text{score}(\mathbf{q}, \mathbf{k}_i))}{\sum_{j=1}^{n} \exp(\text{score}(\mathbf{q}, \mathbf{k}_j))} \quad \text{(softmax normalization}$$

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

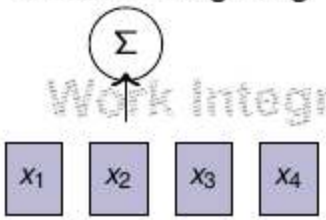
Core Concept: Weighted Combinations

Key Principle: Not all inputs contribute equally – attention learns relevance.

Without Attention:

$$\text{output} = f\left(\sum_{i=1}^{n} x_i\right)$$

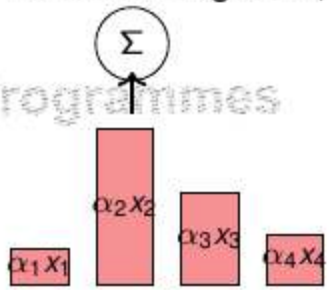
Uniform weighting



With Attention:

$$\text{output} = f\left(\sum_{i=1}^{n} \alpha_i x_i\right)$$

Learned weights α_i



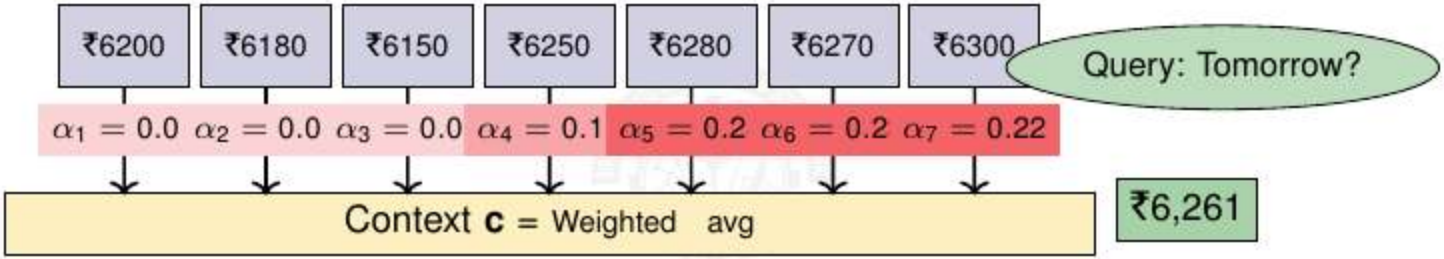
Constraint: Weights sum to 1: $\sum_{i=1}^{n} \alpha_i = 1$ (probability distribution)

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

EXAMPLE 1: STOCK PRICE PREDICTION

Task: Predict tomorrow's gold price (₹/gram) using past 7 days.

Input: Gold prices = [₹6,200, ₹6,180, ₹6,150, ₹6,250, ₹6,280, ₹6,270, ₹6,300]



Observation: Recent days (5-7) receive higher attention weights α_i.

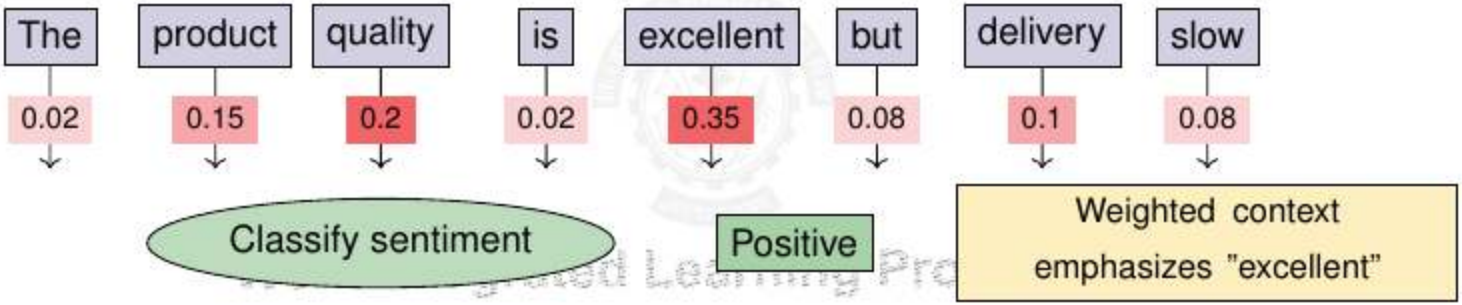
$$\mathbf{c} \approx 0.03 \times 6200 + 0.05 \times 6180 + 0.07 \times 6150 + 0.15 \times 6250 + 0.25 \times 6280 + 0.23 \times 6270 + 0.22 \times 6300$$
$$\approx 6261$$

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

EXAMPLE 2: SENTIMENT ANALYSIS

Task: Classify review sentiment while identifying key phrases.

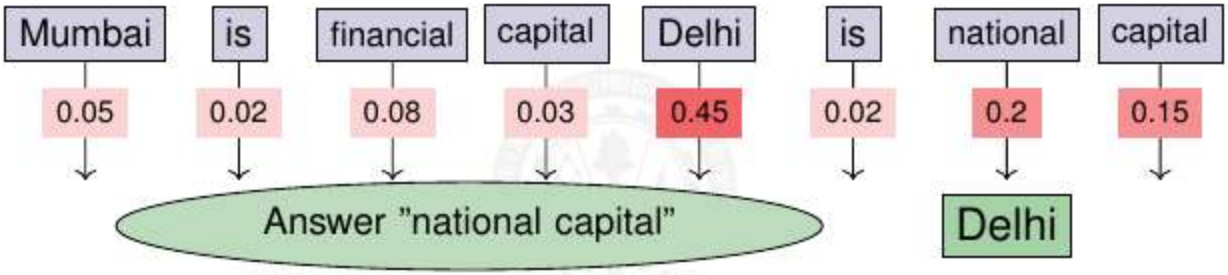
Input: "The product quality is excellent but delivery was very slow"



Key Insight: Model learns "excellent" ($\alpha_5 = 0.35$) is most important for sentiment, despite negative word "slow" present.

EXAMPLE 3: Q& A WITH ATTENTION

Context: "Mumbai is the financial capital. Delhi is the national capital."
Question: "What is the national capital?"



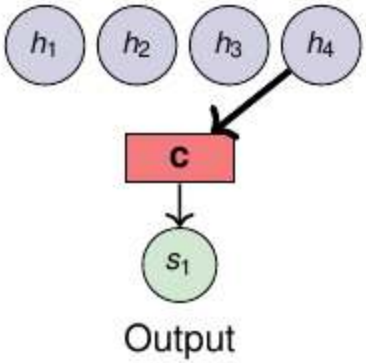
Attention weights show:

- Highest attention on "Delhi" ($\alpha_5 = 0.45$)
- Secondary attention on "national" ($\alpha_7 = 0.20$) and "capital" ($\alpha_4 = 0.15$)
- Model correctly ignores "financial capital"

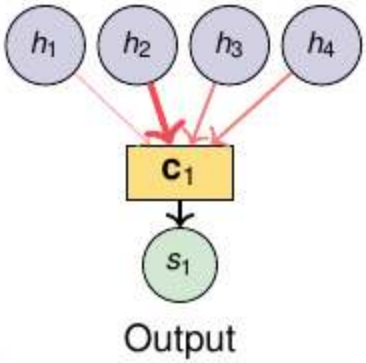
BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Without vs With Attention: Side-by-Side Comparison

Without Attention:



With Attention:



Issues:

- Fixed bottleneck **c**
- Information loss
- Poor long sequences

Benefits:

- Dynamic context $\mathbf{c}_t$
- Selective focus
- Handles long sequences

Key Difference: Context adapts to what decoder needs at each step!

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

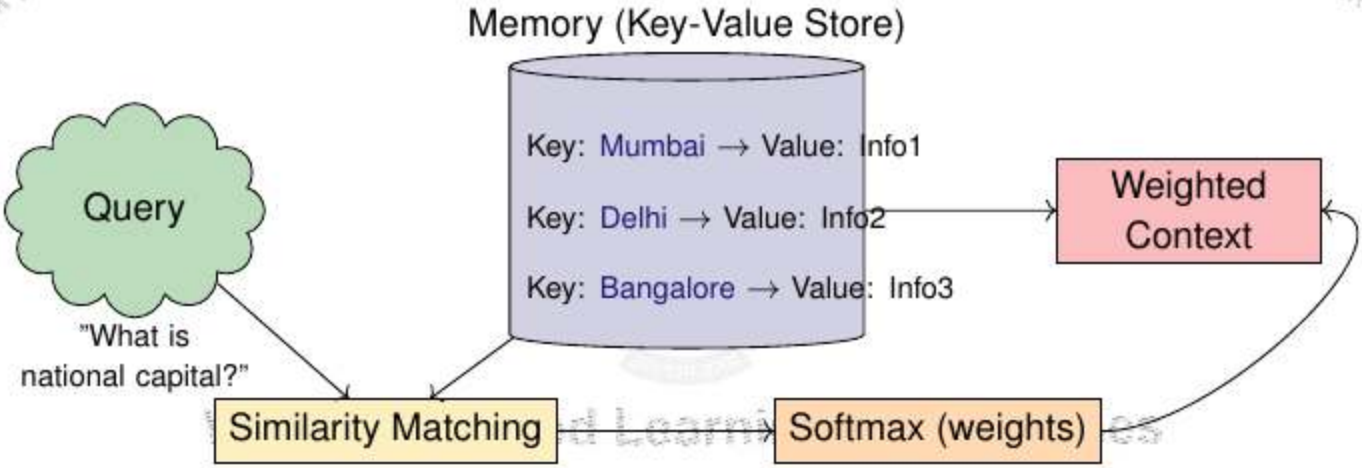
Table of Contents

1. Encoder-Decoder Foundation
2. Introduction to Attention Mechanisms
3. Query-Key-Value Framework
4. Attention Scoring Functions
5. Bahdanau Attention for Seq2Seq
6. Types of Attention
7. Self-Attention & Positional Encoding
8. Summary & Key Takeaways

Work Integrated Learning Programmes

Attention as Information Retrieval

Analogy: Attention works like a database lookup system.



Process:

1. Query searches memory via **key similarity**
2. Compute attention weights based on query-key match
3. Retrieve **values** weighted by attention

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

QUERY-KEY-VALUE (QKV): COMPONENTS

Three distinct roles in attention mechanism:

- Query ($\mathbf{q}$):
  - What information are we seeking?
  - Comes from decoder state: "I need to generate next word"
  - Example: Decoder state when generating "Delhi"
- Keys ($\{\mathbf{k}_i\}$):
  - What's available to match against?
  - Comes from encoder states: representations of input tokens
  - Example: Encoded representations of [Mumbai, is, financial, capital, Delhi, ...]
- Values ($\{\mathbf{v}_i\}$):
  - What actual content do we retrieve?
  - Also from encoder states (often same as keys)
  - Example: Contextual embeddings of input words

Intuition: Query searches Keys, retrieves matching Values.

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

QKV Paradigm: Detailed Mechanics

Step-by-step attention computation:

1. Compute compatibility scores between query and all keys:

$$e_i = \text{score}(\mathbf{q}, \mathbf{k}_i) \quad \text{for } i = 1, \ldots, n$$

2. Normalize with softmax to get attention weights:

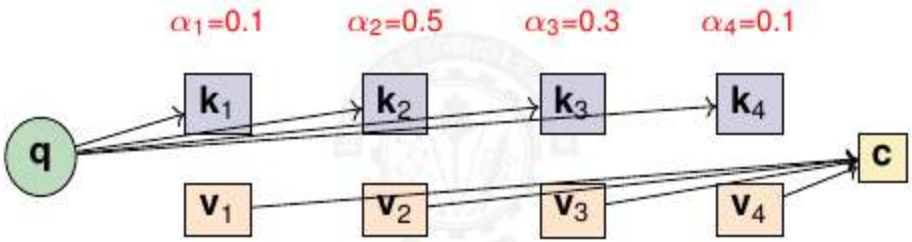
$$\alpha_i = \frac{\exp(e_i)}{\sum_{j=1}^{n} \exp(e_j)} \quad \left(\text{ensures } \sum_i \alpha_i = 1\right)$$

3. Compute weighted sum of values:

$$\mathbf{c} = \sum_{i=1}^{n} \alpha_i \mathbf{v}_i$$

QKV Paradigm: Visual

- Query $\mathbf{q}$: What information are we seeking? Decoder state
- Key $\mathbf{k}_i$: What's available to match against? Encoder state
- Value $\mathbf{v}_i$: What actual content do we retrieve? Encoder state



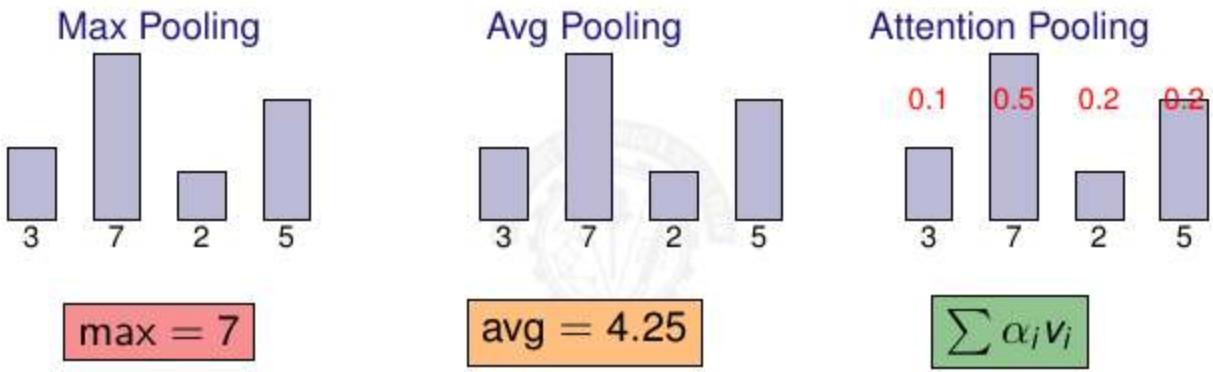
$$\alpha_i = \mathit{softmax}(\text{score}(\mathbf{q}, \mathbf{k}_i)) \quad \text{for } i = 1, \ldots, n$$

$$\mathbf{c} = \sum_{i=1}^{n} \alpha_i \mathbf{v}_i$$

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Attention as Weighted Pooling

Pooling Perspective: Attention is soft, adaptive pooling over sequence.



Key Difference:

- Max/Avg pooling: Fixed, uniform aggregation
- Attention: Data-dependent, learned aggregation based on query

Table of Contents

1. Encoder-Decoder Foundation
2. Introduction to Attention Mechanisms
3. Query-Key-Value Framework
4. Attention Scoring Functions
5. Bahdanau Attention for Seq2Seq
6. Types of Attention
7. Self-Attention & Positional Encoding
8. Summary & Key Takeaways

Work Integrated Learning Programmes

Dot-Product Attention

Simplest form: Measure similarity via inner product.

$$\text{score}(\mathbf{q}, \mathbf{k}_i) = \mathbf{q}^\top \mathbf{k}_i = \sum_{j=1}^{d} q_j k_{i,j}$$

$$\text{Attention } \alpha_i = \frac{\exp(\mathbf{q}^\top \mathbf{k}_i)}{\sum_{j=1}^{n} \exp(\mathbf{q}^\top \mathbf{k}_j)} = \text{softmax}(\mathbf{q}^\top \mathbf{K}^\top)_i$$

where $\mathbf{K} = [\mathbf{k}_1, \ldots, \mathbf{k}_n]^\top \in \mathbb{R}^{n \times d}$.

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

EXAMPLE: DOT-PRODUCT ATTENTION

Example: Stock price prediction

- Query: $\mathbf{q} = [0.8, 0.6]^\top$ (tomorrow's expected trend features)
- Keys: $\mathbf{k}_1 = [0.5, 0.3]^\top$, $\mathbf{k}_2 = [0.9, 0.7]^\top$, $\mathbf{k}_3 = [0.4, 0.2]^\top$
- Scores:

$$
\begin{aligned}
e_1 &= 0.8(0.5) + 0.6(0.3) = 0.58 \\
e_2 &= 0.8(0.9) + 0.6(0.7) = 1.14 \quad \text{(highest!)} \\
e_3 &= 0.8(0.4) + 0.6(0.2) = 0.44
\end{aligned}
$$

- Weights: $\alpha_1 = 0.21$, $\alpha_2 = 0.62$, $\alpha_3 = 0.17$ (after softmax)
- Interpretation: Day 2 most aligned with query $\rightarrow$ highest attention.

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Scaled Dot-Product Attention

Problem: For large dimension d, dot products grow large $\rightarrow$ softmax saturates.
Solution (Vaswani et al., 2017): Scale by $\sqrt{d_k}$ to normalize variance.

$$\text{score}(\mathbf{q}, \mathbf{k}) = \frac{\mathbf{q}^\top \mathbf{k}}{\sqrt{d_k}}$$

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

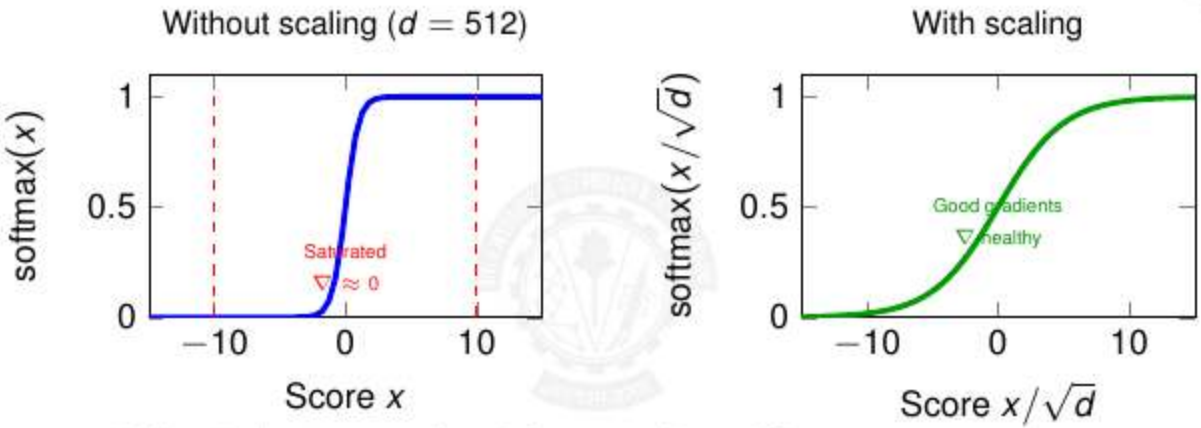
Benefits:

- Keeps scores in reasonable range regardless of dimension
- Prevents vanishing gradients in softmax
- Standard in all Transformer models

Work Integrated Learning Programmes

Why Scaling Matters: Gradient Flow

Effect of scaling on softmax gradients:



Impact:

- Left: Large scores push softmax to extremes (0 or 1)
- Right: Scaled scores maintain gradient flow
- Critical for training deep networks (6-24+ layers)

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Additive Attention (Bahdanau, 2014)

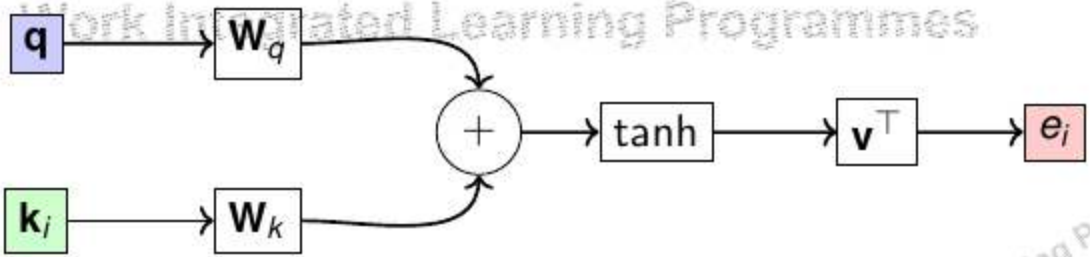
- Original attention mechanism for neural machine translation.

$$\text{score}(\mathbf{q}, \mathbf{k}_i) = \mathbf{v}^\top \tanh(\mathbf{W}_q \mathbf{q} + \mathbf{W}_k \mathbf{k}_i)$$

where:

- $\mathbf{W}_q \in \mathbb{R}^{h \times d_q}$: Query projection matrix
- $\mathbf{W}_k \in \mathbb{R}^{h \times d_k}$: Key projection matrix
- $\mathbf{v} \in \mathbb{R}^h$: Learnable weight vector
- h: Hidden dimension (hyperparameter, e.g., 256)

- Computation flow:



- Advantage: Can handle $d_q \neq d_k$ (different query/key dimensions).

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Comparison: Dot-Product vs Additive

Aspect	Dot-Product (Scaled)	Additive (Bahdanau)
Formula	$\frac{\mathbf{q}^\top \mathbf{k}}{\sqrt{d_k}}$	$\mathbf{v}^\top \tanh(\mathbf{W}_q\mathbf{q} + \mathbf{W}_k\mathbf{k})$
Parameters	None	$\mathbf{W}_q, \mathbf{W}_k, \mathbf{v}$
Complexity	$O(d)$ per pair	$O(d \cdot h + h)$ per pair
Expressiveness	Limited to linear	Nonlinear (tanh)
Parallelization	Excellent (matrix mult)	Moderate
Dimension req.	Requires $d_q = d_k$	Works with $d_q \neq d_k$
Modern usage	Transformers	Early seq2seq

Historical note: Additive was better for RNN-based models (2014-2016), but scaled dot-product dominates in Transformer era (2017+).

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Computational Complexity Analysis

For sequence length n, dimension d, and m queries:

Operation	Scaled Dot-Product	Additive
Score computation	$O(mnd)$	$O(mndh + mnh)$
Softmax	$O(mn)$	$O(mn)$
Weighted sum	$O(mnd)$	$O(mnd)$
Total	$\mathbf{O(mnd)}$	$\mathbf{O(mndh)}$

Matrix formulation (batch processing):

- Scaled dot-product: $\mathbf{QK}^\top \in \mathbb{R}^{m \times n}$ computed in one GPU operation
- Highly parallelizable on modern hardware (GPUs/TPUs)
- Memory efficient: $O(mn)$ for attention matrix

Practical impact: Dot-product is 3-5× faster in practice due to better hardware utilization.

Scoring Functions: Overview

The scoring function score($\mathbf{q}, \mathbf{k}$) measures query-key compatibility.

Method	Score Function	Parameters
Dot Product	$\mathbf{q}^\top \mathbf{k}$	None
Scaled Dot Product	$\frac{\mathbf{q}^\top \mathbf{k}}{\sqrt{d_k}}$	None
Additive (Bahdanau)	$\mathbf{v}^\top \tanh(\mathbf{W}_q \mathbf{q} + \mathbf{W}_k \mathbf{k})$	$\mathbf{W}_q, \mathbf{W}_k, \mathbf{v}$
Multiplicative (Luong)	$\mathbf{q}^\top \mathbf{W} \mathbf{k}$	$\mathbf{W}$

Design Trade-offs:

- Dot product: Fast (matrix ops), no parameters, requires $d_q = d_k$
- Scaled Dot Product : Prevents saturation for large dimensions (Transformer default)
- Additive: More expressive, flexible dimensions, more parameters

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

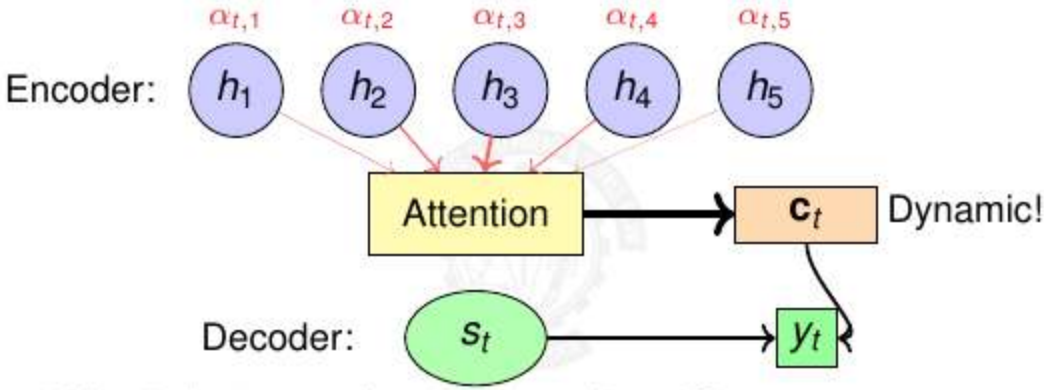
Table of Contents

1. Encoder-Decoder Foundation
2. Introduction to Attention Mechanisms
3. Query-Key-Value Framework
4. Attention Scoring Functions
5. Bahdanau Attention for Seq2Seq
6. Types of Attention
7. Self-Attention & Positional Encoding
8. Summary & Key Takeaways

Work Integrated Learning Programmes

Bahdanau Attention: The Breakthrough

Key Innovation (Bahdanau et al., 2014): Replace fixed context with dynamic attention.



Context at time t:

$$\mathbf{c}_t = \sum_{i=1}^{T} \alpha_{t,i} \mathbf{h}_i$$

where $\alpha_{t,i}$ depends on current decoder state s_{t-1} and encoder state h_i.

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Bahdanau Attention: Math

At each decoder time step t:

- Step 1: Compute alignment scores
- Step 2: Normalize to attention weights
- Step 3: Compute context vector
- Step 4: Update decoder state
- Step 5: Generate output distribution

$$e_{t,i} = \mathbf{v}^\top \tanh(\mathbf{W}_s s_{t-1} + \mathbf{W}_h h_i)$$

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^{T} \exp(e_{t,j})}$$

$$\mathbf{c}_t = \sum_{i=1}^{T} \alpha_{t,i} \mathbf{h}_i$$

$$s_t = \text{RNN}([y_{t-1}; \mathbf{c}_t], s_{t-1})$$

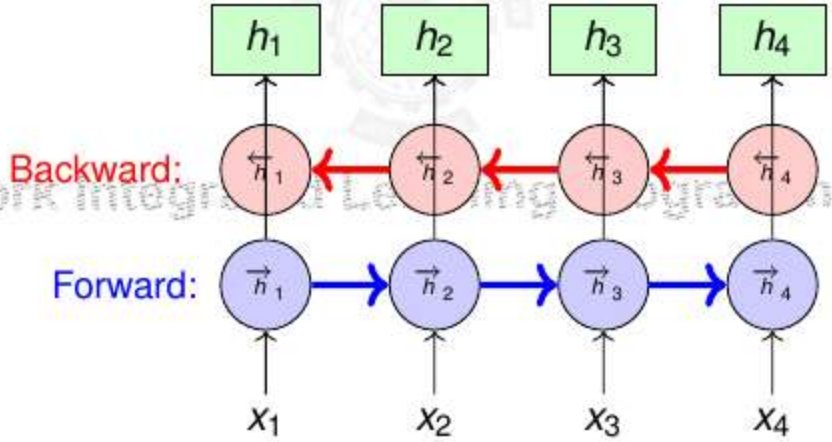
$$P(y_t | y_{<t}, \mathbf{x}) = \text{softmax}(\mathbf{W}_o [s_t; \mathbf{c}_t])$$

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Encoder with Attention: Bidirectional RNN

Bahdanau used bidirectional RNN encoder to capture context from both directions.

$$\overrightarrow{h}_i = \text{RNN}_{\text{fwd}}(x_i, \overrightarrow{h}_{i-1})$$
$$\overleftarrow{h}_i = \text{RNN}_{\text{bwd}}(x_i, \overleftarrow{h}_{i+1})$$
$$h_i = [\overrightarrow{h}_i; \overleftarrow{h}_i]$$



Benefit: h_i encodes both past and future context for position i.

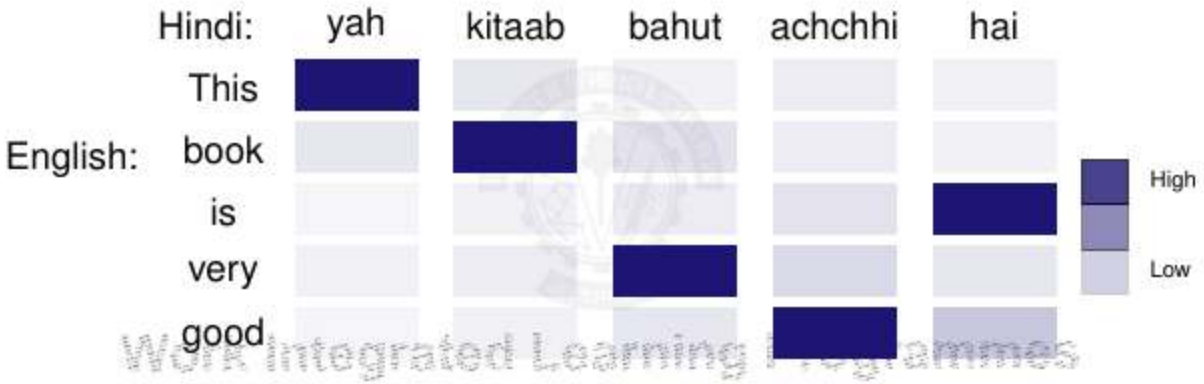
BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

ATTENTION ALIGNMENT: VISUALIZATION

Example: Hindi to English translation

Source (Hindi): yah kitaab bahut achchhi hai

Target (English): This book is very good



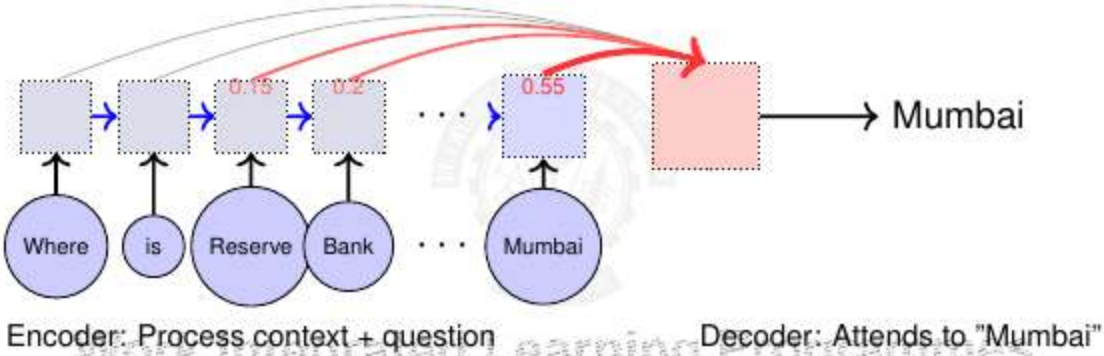
Observations:

- Strong diagonal pattern: word-level alignment
- "yah" $\leftrightarrow$ "This": $\alpha = 0.85$
- "kitaab" $\leftrightarrow$ "book": $\alpha = 0.82$

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

EXAMPLE: Q& A WITH ATTENTION

Context: "Mumbai is the financial capital. The Reserve Bank is located there."
Question: "Where is Reserve Bank?"
Answer: "Mumbai"



Attention interpretation:

- Decoder focuses on context mentioning "Reserve Bank" and "Mumbai"
- Highest weight ($\alpha = 0.55$) on "Mumbai" in context
- Successfully extracts answer from relevant part of input

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Implementation Considerations

- Training with Attention:
  - Teacher forcing: Use ground-truth y_{t-1} as decoder input
  - Loss function: Cross-entropy over target vocabulary

$$\mathcal{L} = -\sum_{t=1}^{T'} \log P(y_t^* | y_{<t}^*, \mathbf{x})$$

  - Optimizer: Adam with learning rate warmup and decay
  - Regularization: Dropout on encoder/decoder states and attention weights
- Inference with Attention:
  - Greedy decoding: Select $\arg\max_y P(y_t | y_{<t}, \mathbf{x})$ at each step
  - Beam search: Maintain top-k hypotheses (typical $k = 5$)
  - Attention recomputed: Fresh context $\mathbf{c}_t$ at each decoder step
  - Stopping: Generate until $\langle$END$\rangle$ token or max length
- Computational cost:
  - Attention: $O(T \times T')$ for input length T, output length T'

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Table of Contents

1. Encoder-Decoder Foundation
2. Introduction to Attention Mechanisms
3. Query-Key-Value Framework
4. Attention Scoring Functions
5. Bahdanau Attention for Seq2Seq
6. Types of Attention
7. Self-Attention & Positional Encoding
8. Summary & Key Takeaways

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Attention Types: Taxonomy

Type	Q Source	K,V Source	Use Case
Self-Attention	Same sequence	Same sequence	Intra-sequence relationships
Cross-Attention	Sequence A	Sequence B	Inter-sequence alignment (seq2seq)
Masked Attention	Same sequence	Same (causal mask)	Autoregressive generation
Multi-Head	Same (parallel)	Same (parallel)	Multiple relationship types

Key Distinctions:

- Source of Q, K, V: Where do query, key, value vectors come from?
- Masking: What positions can attend to what?
- Purpose: What relationships are being modeled?

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

SELF-ATTENTION: DEFINITION AND PURPOSE

- Self-attention: Query, key, and value all come from the *same* sequence.

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V$$

  where $\mathbf{X} = [x_1, \ldots, x_n]^\top$ is the input sequence.
- Purpose: Model relationships *within* a sequence.
- Time Series Example: Stock price prediction using past 24 hours
  - Input: Gold prices over 24 hours: [₹6,180, ₹6,200, . . ., ₹6,280]
  - Self-attention allows each hour to attend to all other hours
  - Example: Hour 24 might attend strongly to hours 17, 10, 3 (7-hour periodicity)
  - Captures temporal patterns automatically
- Text Example: "The student didn't pass the exam because he was unprepared"
  - Self-attention helps "he" attend to "student"
  - Resolves coreference within the sentence
  - Captures long-range dependencies ("he" → "student")

Cross-Attention: Inter-Sequence Alignment

- Cross-attention: Query from one sequence, keys and values from *another*.

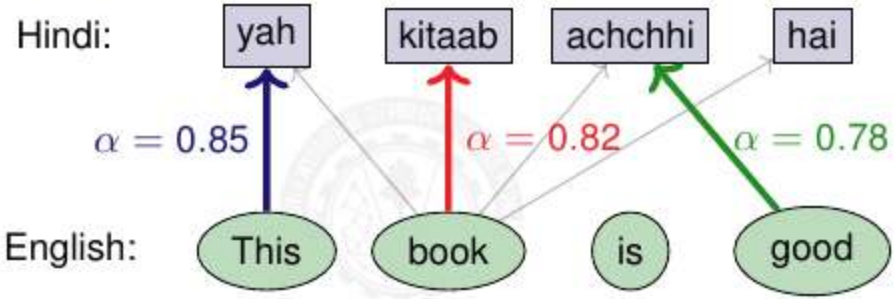
$$\mathbf{Q} = \mathbf{X}_A \mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}_B \mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}_B \mathbf{W}_V$$

- Time Series Example: Multivariate forecasting
  - Sequence A: Stock prices (query current state)
  - Sequence B: Economic indicators (keys/values: inflation, GDP, interest rates)
  - Cross-attention: Stock prices attend to relevant economic indicators
  - Example: Gold price queries inflation rate and USD exchange rate
- Text Example: Hindi $\rightarrow$ English translation
  - Sequence A (Decoder): English words being generated
  - Sequence B (Encoder): Hindi source sentence
  - When generating "book", decoder attends to Hindi "kitaab"
  - Example: Generating "yah kitaab achchhi hai" $\rightarrow$ "This book is good"
    - ★ "book" (EN) attends to "kitaab" (HI) with $\alpha \approx 0.85$
    - ★ "good" (EN) attends to "achchhi" (HI) with $\alpha \approx 0.78$

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Cross-Attention: Visualization

- Machine Translation Example: Hindi to English



- Key: Decoder (English) dynamically attends to relevant parts of encoder (Hindi).
- This is exactly Bahdanau attention – the original cross-attention mechanism!

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

MASKED ATTENTION: CAUSAL/AUTOREGRESSIVE

- Masked attention: Self-attention with *causal constraint* – can only attend to past.

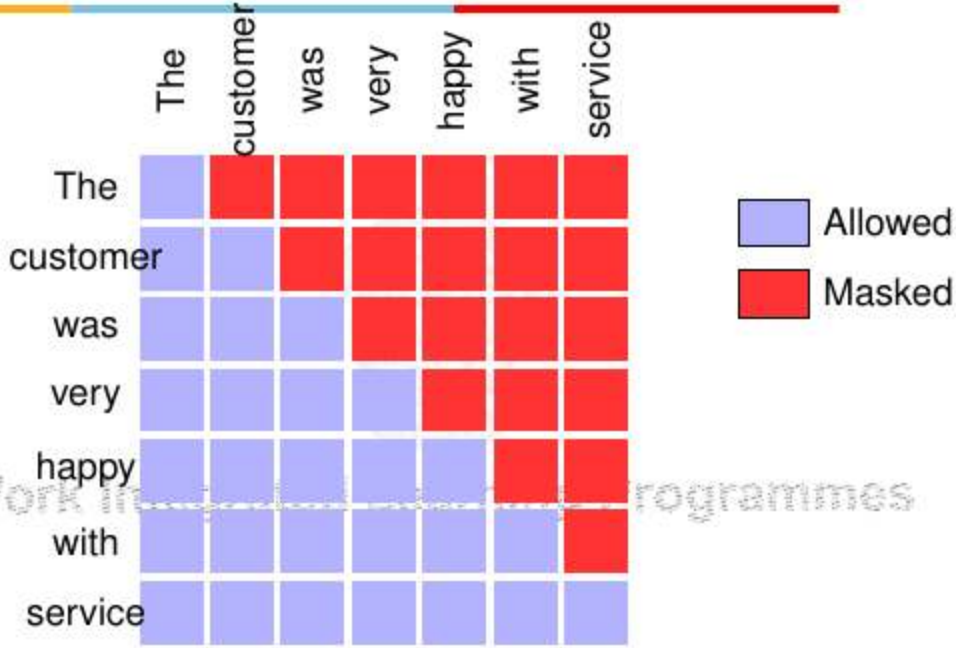
$$\text{Mask}_{ij} = \begin{cases} 0 & \text{if } j \leq i \quad \text{(allowed)} \\ -\infty & \text{if } j > i \quad \text{(masked)} \end{cases}$$

- Time Series Example: Stock price prediction (prevent future leakage)
  - Predicting price at time t using only times $1, \ldots, t-1$
  - Mask prevents attending to future times $t+1, t+2, \ldots$
  - Ensures no "cheating" during training
  - Example: Predict tomorrow's gold price without seeing tomorrow's actual price
- Text Example: GPT-style language modeling
  - Task: Predict next word given previous words. "The customer was very ___"
  - Token at position 5 ("very") can only attend to positions 1-4
  - Cannot see future tokens during training or inference
  - Autoregressive: generate one token at a time, left-to-right

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Masked Attention: Causal Mask Visualization

- Attention mask pattern: Query (row) attends to Key (column)



- Interpretation: Lower triangular pattern ensures token i only sees tokens $\leq i$.
- Implementation: Add $-\infty$ to masked positions before softmax $\rightarrow$ weight $= 0$.

Multi-Head Attention: Concept

- Motivation: Single attention may miss different types of relationships.
- Solution: Use multiple attention "heads" in parallel, each learning different patterns.
- Time Series Example: Gold price with 3 heads
  - Head 1: Attends to short-term trends (last 2-3 days)
  - Head 2: Attends to weekly patterns (7-day cycles)
  - Head 3: Attends to monthly patterns (30-day trends)
  - Combined: Richer representation than single attention
- Text Example: Sentence with 3 heads
  - Syntax head: Subject-verb-object relationships
    - "The customer complained" – verb attends to subject
  - Semantic head: Meaning and coreference
    - "he" attends to "customer" (resolves pronoun)
  - Positional head: Adjacent word dependencies
    - "very happy" – modifier-adjective pair

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Multi-Head Attention: Architecture

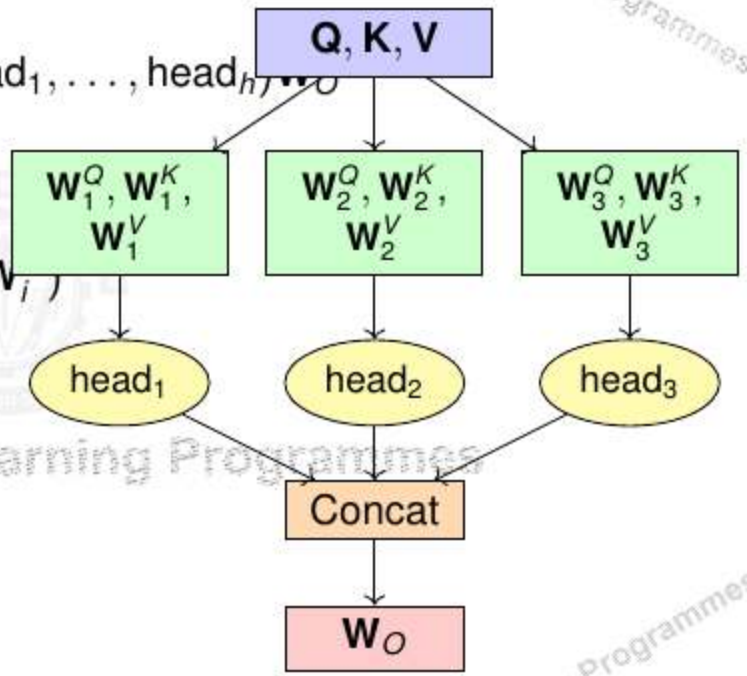
- Formula (Vaswani et al., 2017):

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\mathbf{W}_O$$

where each head i computes:

$$\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V)$$

- Typical hyperparameters:
  - $h = 8$ heads
  - $d_{\text{model}} = 512$
  - $d_k = d_v = 64$ per head



Multi-head output

When to Use Which Attention Type?

Attention Type	Use When...
Self-Attention	Modeling relationships within a single sequence Building contextual representations (e.g., BERT encoder) Capturing long-range dependencies in text or time series
Cross-Attention	Aligning two different sequences (seq2seq) Translation, summarization, question answering Transformer decoder attending to encoder outputs
Masked	Autoregressive generation (language modeling) Preventing information leakage from future GPT-style models, time series forecasting
Multi-Head	Capturing diverse relationship types simultaneously All modern Transformers (BERT, GPT, T5, ViT) When you need multiple attention patterns

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Table of Contents

1. Encoder-Decoder Foundation
2. Introduction to Attention Mechanisms
3. Query-Key-Value Framework
4. Attention Scoring Functions
5. Bahdanau Attention for Seq2Seq
6. Types of Attention
7. Self-Attention & Positional Encoding
8. Summary & Key Takeaways

Work Integrated Learning Programmes

Self-Attention: Formal Definition

- Self-attention allows each position to attend to all positions in the *same* sequence.
- Given input $\mathbf{X} = [\mathbf{x}_1, \ldots, \mathbf{x}_n] \in \mathbb{R}^{n \times d}$:
  1. Linear projections:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V$$

  2. Scaled dot-product attention:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

  3. Output:

$$\mathbf{Z} = \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \in \mathbb{R}^{n \times d_v}$$

- Key property: Every output $\mathbf{z}_i$ is a weighted combination of *all* inputs $\{\mathbf{x}_j\}_{j=1}^n$.
- Difference from RNN: Direct connections between all positions (no sequential bottleneck).

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

CNNs vs RNNs vs Self-Attention

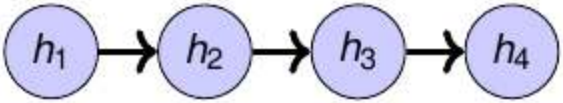
Aspect	CNN	RNN	Self-Attn
Receptive field	Local (kernel k)	Sequential (history)	Global (all)
Path length	$O(\log n)$	$O(n)$	$O(1)$
Parallelization	High	Sequential	High
Complexity	$O(k \cdot n \cdot d^2)$	$O(n \cdot d^2)$	$O(n^2 \cdot d)$
Long-range	Many layers	Vanishing grad	Direct
Position info	Implicit	Built-in	Need encodin

- Key insight: Self-attention provides $O(1)$ path between any two positions!
- Trade-off: Better long-range modeling but $O(n^2)$ memory and computation.

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

PARALLELIZATION: RNN VS SELF-ATTENTION

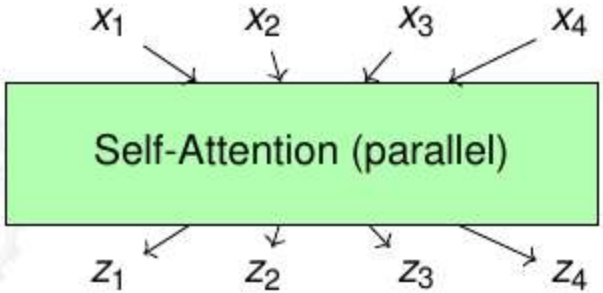
RNN (Sequential):



Must compute sequentially

Bottleneck: h_4 depends on h_3 depends on h_2 depends on h_1

Self-Attention (Parallel):



All positions in parallel

Benefit: All outputs computed simultaneously on GPU!

Result: Transformers train 10-100× faster than RNNs for long sequences.

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Position-Invariance Problem

- Critical issue: Self-attention is *permutation-invariant*!

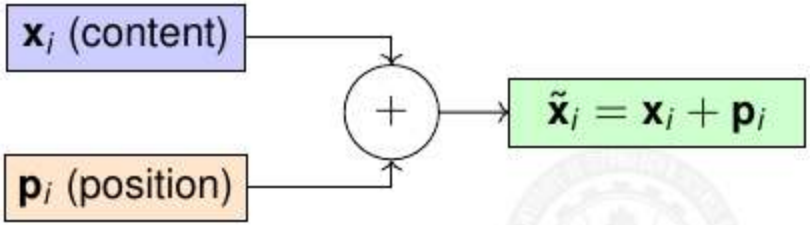
$$\text{Attention}(\mathbf{X}) = \text{Attention}(\mathbf{PX})$$

  for any permutation matrix $\mathbf{P}$.
- Example: These sentences are treated identically:
  - Original: "The customer was happy with service"
  - Shuffled: "service with happy was customer The"
- Why? Attention weights depend only on *content*, not *position*.
- Problem for NLP: Word order matters!
  - "Dog bites man" $\neq$ "Man bites dog"
  - Need position information to distinguish
- Solution: Inject positional information into input embeddings.

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Positional Encoding: The Solution

Add position information to input embeddings:



Input to Transformer

Two approaches:

1. Learned positional embeddings (BERT, GPT-2):
   - $\mathbf{p}_i$ is a learned parameter for each position i
   - Simple, but doesn't generalize beyond training length
2. Fixed sinusoidal encoding (Original Transformer):
   - $\mathbf{p}_i$ computed using sine/cosine functions
   - Deterministic, generalizes to any sequence length

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Absolute Positional Encoding: Sinusoidal

Vaswani et al. (2017): Use fixed sinusoidal functions.

$$\mathrm{PE}(\mathrm{pos}, 2i) = \sin\left(\frac{\mathrm{pos}}{10000^{2i/d_{\mathrm{model}}}}\right)$$

$$\mathrm{PE}(\mathrm{pos}, 2i+1) = \cos\left(\frac{\mathrm{pos}}{10000^{2i/d_{\mathrm{model}}}}\right)$$

- pos: Position in sequence $(0, 1, 2, \ldots)$
- i: Dimension index $(0, 1, \ldots, d_{\mathrm{model}}/2 - 1)$
- d_{model}: Model dimension (e.g., 512)

Intuition:

- Different dimensions use different frequencies
- Lower dimensions = lower frequency (long-range patterns)
- Higher dimensions = higher frequency (fine-grained positions)
- Each position gets unique encoding vector

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Why Sinusoidal Encoding Works

- Key mathematical property: For any fixed offset k, $\mathrm{PE}(\mathrm{pos}+k)$ is a *linear function* of $\mathrm{PE}(\mathrm{pos})$.

$$\mathrm{PE}(\mathrm{pos}+k, 2i) = \mathbf{M}_k \cdot \mathrm{PE}(\mathrm{pos}, 2i) \quad \text{for some transformation matrix } \mathbf{M}_k$$

- Trigonometric identity:

$$\sin(\alpha+\beta) = \sin\alpha\cos\beta + \cos\alpha\sin\beta$$
$$\cos(\alpha+\beta) = \cos\alpha\cos\beta - \sin\alpha\sin\beta$$

- Example: For dimension i, let $\omega_i = 1/10000^{2i/d}$

$$\begin{aligned}\mathrm{PE}(\mathrm{pos}+k, 2i) &= \sin((\mathrm{pos}+k)\omega_i) \\ &= \sin(\mathrm{pos}\omega_i)\cos(k\omega_i) + \cos(\mathrm{pos}\omega_i)\sin(k\omega_i)\end{aligned}$$

This is a linear combination of $\mathrm{PE}(\mathrm{pos}, 2i)$ and $\mathrm{PE}(\mathrm{pos}, 2i+1)$.

- Benefit: Model can learn to attend by *relative position* (e.g., "3 positions back").

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Table of Contents

1. Encoder-Decoder Foundation
2. Introduction to Attention Mechanisms
3. Query-Key-Value Framework
4. Attention Scoring Functions
5. Bahdanau Attention for Seq2Seq
6. Types of Attention
7. Self-Attention & Positional Encoding
8. Summary & Key Takeaways

Work Integrated Learning Programmes

Summary: Attention Mechanisms

1. Attention Mechanism
   - Dynamic, weighted aggregation based on relevance
   - Query-Key-Value framework for information retrieval
   - Solves bottleneck by attending selectively
2. Scoring Functions
   - Dot-product: simple, fast
   - Scaled dot-product: prevents saturation (Transformer default)
   - Additive: more expressive, slower
3. Attention Types
   - Self: intra-sequence, Cross: inter-sequence
   - Masked: causal, Multi-head: diverse patterns
4. Positional Encoding
   - Required for position-aware self-attention
   - Sinusoidal: fixed, generalizable

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Key Formulas to Remember

- 1. Attention (general):

$$\text{Attention}(\mathbf{q}, \{\mathbf{k}_i, \mathbf{v}_i\}) = \sum_{i=1}^{n} \alpha_i \mathbf{v}_i \quad \text{where} \quad \alpha_i = \text{softmax}\left(\text{score}(\mathbf{q}, \mathbf{k}_i)\right)$$

- 2. Scaled Dot-Product Attention:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

- 3. Multi-Head Attention:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\mathbf{W}_O$$

- 4. Positional Encoding:

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d}}\right), \quad \text{PE}(\text{pos}, 2i+1) = \cos\left(\frac{\text{pos}}{10000^{2i/d}}\right)$$

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Work Integrated Learning Programmes

Thank you :)
All the best :)

Work Integrated Learning Programmes